Nate Capistrant - Fossa
Senior Project

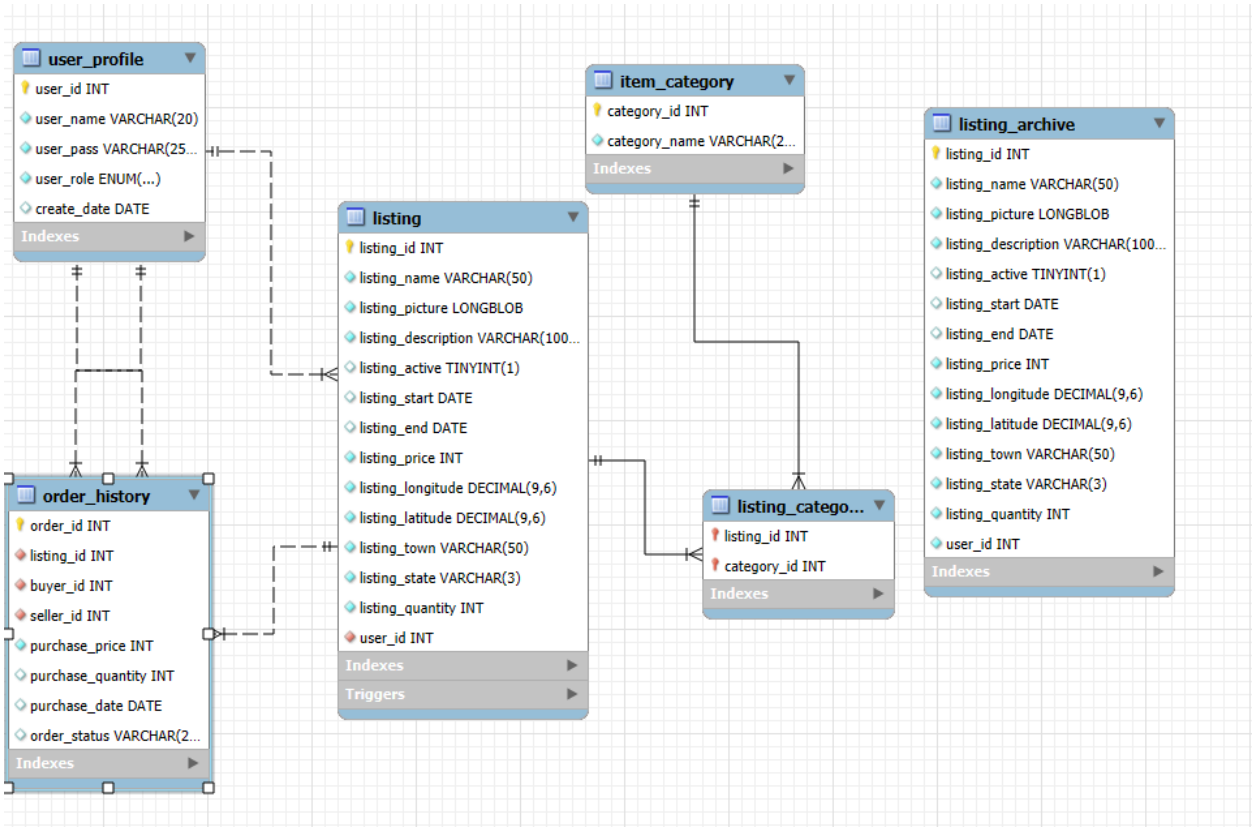
Introduction

My project is an ecommerce platform that is supposed to mimic platforms such as facebook marketplace or ebay (without bidding).

Project Design and Components

My project is built using Java for the application layer and JavaFX for the graphical layer. Lastly my backend layer is built with a mySQL server. I will start off by giving a quick overview of the backend. Below is a picture of the database design. The ‘user_profile’ table is to store user accounts, ‘listing’ is used to hold the listings. The ‘listing’ table will store the owner as a foreign key. In this case there was no need for a joint table, but if a listing could have multiple collaborators (sellers) then a joint table would have been necessary. My ‘order_history’ table keeps track of the listing that was bought, who bought it and who sold it. The last category that I want to mention is the ‘listing_archive’ table. This is completely optional and can easily be removed. In a real world system this would be its own database all together. The reason I have added it to my project is because on ebay there is a feature that allows you to see your old listings whether they have sold or not. In addition you can ‘re-post’ the exact same listing with all of the same information other than the ending and start date. After a period of time you will not be able to see these listings anymore. I added the listing archive to remove listings from the listing table once they have been inactive for 3 months. Having this archive table is really helpful

in collecting data. You will have all of the information in one section without having the storage interfere with your main server. The last thing I want to note about my database is that in this program I have stored the listing pictures as LONGBLOB in the database. I know that in real world applications this is very bad design as images take up LOTS of space. Ideally these would be stored in their own separate storage module.



Now for the main component the application layer. I am using a mixture of two structural design patterns. The MVC (Model-View-Controller) design pattern, and the DAO (Data Access Object) design pattern. The MVC simply splits up the program into what the user will see (View), the data (Model) and what connects these two components together (Controller). This design pattern is completed well by the DAO design pattern. The DAO would fit in between the model and controller. With DAO you have two classes: your DAO object and normal object. When the object requests information from the database it will not request it directly, instead it

will call the DAO to get that data for it adding a level of abstraction which will help see when an error occurs. I use two APIs in this project. The first is google maps. I use google maps two different times in the project. The first function is the user will enter the location of their item, and then use Google's geocoding function to turn the string literal address into a longitude and latitude coordinate. These coordinates will not only validate that there is an address there but also keep an accurate location in case something was incorrectly entered. Ex. If I were to type '134 main rd' instead of '123 main st' google maps generally will be able to notice the mistake and correct it. At times it will assign the complete wrong address, there isn't much I can do about this though, this will be talked about more in depth in the 'Issues Faced Along The Way' section. The second use of this API was to generate map images. Once the user purchases an item, they will be able to see the item address and open a google map window to see where this listing is. My last API is Stripe, which is used to process payments. Currently I have the API generating a client secret upon a user checking out, sending a request for payment without actually creating a payment. There is no way for a user to enter real payment information as I felt there was no need for this in a school project. I didn't know that there was a sandbox mode for the stripe API and will go further into detail in the 'Issues Faced Along The Way' section. Lastly there are the dependencies for the project. I used the following dependencies javafx-fxml, javafx-web, javafx-swing, mysql-connector-j, jbcrypt, gson, json, google-guava, and stripe-java (this is ignoring j-unit as it's not a requirement for users).

There is not much to discuss with the GUI. I used the javafx libraries mostly, in addition to this I created two unique CSS style sheets to style everything. I did use AI to help create these CSS style sheets as they are purely visual and serve little to impact on the system as a whole.

User Manual

Users MUST be connected to the internet. After giving a presentation on 04/28/2026 I found this out the hard way. While some functionality will work without the internet users will be unable to use both APIs. Stripe will not be able to connect the program nor will Google Maps be able to validate user addresses, or produce the map picture. Now to start this program users will open the app. They will be brought to the login screen. If they do not already have an account a popup menu will appear for them to create their account. For an account to be created they must enter a unique user name, and a password that meets our minimum password standard. Upon successfully creating their account they can log in. Once logging into their account they will be brought to the main menu. The main menu is where all listings (except their own) will be posted. From this screen users can either look at their profile, create their own listing or browse through listings. They will be unable to go to the checkout page as they have yet to add an item to their cart. Carts have an unlimited size and users could potentially add everything to their cart. If a user is to click on a listing they will be brought to a page that shows them every detail about the listing. The name, full description, product image, how much it is per item, and how many items are available. Some listings may only have one item available but if a seller can sell multiple items. For instance if I have 4 of the same book I can add a quantity of 4 when creating the listing. The location for the item will not be shown at this time. All the user will be able to see is the item state, and town. It would be dangerous for the seller if the item location was public to anyone and could potentially lead to theft. This is why the exact coordinates/map image is only visible once you purchase the item. After any amount of items has been added to the cart you will finally be able to check out. Additionally once adding a listing to the cart, that listing

will no longer be visible to the user. This is to stop duplicate entries into their cart. Another way I could have done this was by doing a quick search and seeing if that item already exists in their cart. But ultimately I decided on this choice as it would be less debugging overall. Once reaching the checkout screen all of the items in the users cart will be displayed in a listview. They can review each item seeing how much their total is. They will also be able to adjust how many they actually want to purchase here. If a user added 10 of one item and saw how much their total was, they can adjust it accordingly. The only limitation is the user will be unable to go over the max quantity an item had upon entering the cart. So if a listing were to have a maximum of 5 items for sale, the user would be unable to purchase more than 5 of that listing. The last thing to note about the checkout screen is that if the database were to be changed in different ways the checkout will catch this. Examples being if a listing has 5 quantities available, and a user adds all 5 to their cart. Before they are able to check out another user buys 2 of that same item only leaving 3 available. When trying to check out the user will be unable to purchase 5. The Stripe API is not called meaning no charge will happen for this item (this is still true if the user had multiple items in their cart, they will be charged only for valid sales). The screen will immediately update telling the user there were not long 5 items left updating their cart to the new maximum quantity, in this case 3. In another situation we will say all details are the same except now the secondary user bought all of them items. So you still have the listing in your cart, but it’s now unavailable. When trying to check out again it will not purchase this item. It will instead remove the item from your cart alerting you it was removed and telling you that it was removed due to there being no stock left. This ensures that users won't be charged for items they cannot receive. Another option for users is to make their own listings. Clicking the create listing button on the main screen will bring them to the listing screen. Every listing must follow the same

format. Have an ending date within 3 days of today, have a name <50 characters, have a description <1000 characters, have a valid product image, have a valid address, some price <=1, some quantity <=1, and a category. For the case of my project every listing can only have one category, but in real world systems this wouldn't be true and a listing could have multiple categories. Once creating your listing you're brought back to the main screen. Upon clicking the profile screen you will be brought to that screen. The profile screen has a list of all the users active listings. This is where users will be able to edit or remove their listings. There are an additional two buttons on this screen as well. A sales history tab and a purchase history tab. As both names imply users will be able to see their own sales history and or purchase history. Both screens have a generate receipt button on them which creates a .josn file on their homescreen. This .json file will be named after the user account, and what type of receipt it is, sales history or purchase history. I choose .json over csv purely for the fact I have never interacted with a .json file before this project. In the order history tab you will also be able to see the image of where the item location is. To get this you just need to double click on the location of the listing you are trying to see.

Issues Faced Along The Way

The first of my large issues was the Google Maps API and javafx's web view. Originally I wanted to have a popup that gave you a live interactable map of where the item was. But this seems to be nearly impossible using javafx's webview. From my extensive hours researching and debugging javafx's web-kit is highly outdated, and Google Maps sees that browser as a security issue. So nothing would pop up, and the map would say something along the lines of 'no image

could be found.' I even found a way to inject the current version of chrome into the webview window (as it's just a chromium based browser), but this didn't solve the problem either. The only solution I could find online was to create my own Java Chromium Embedded Framework (JCEF). This would allow me to create my own chromium browser to display the map. I believe doing this would have been beyond the scope of this project and could have been a project of its own. So instead of doing this I came up with a compromise. Instead of showing a live physical map I chose to display a static image of the location. This doesn't give nearly as much information to the user but at least it will give them the general idea.

My other major issue during my project was my payment API. Originally I wanted to use the PayPay API. For this I needed a PayPal developer account, so I tried to sign up for one. Every time I attempted to sign up for a developer account I was redirected to default PayPal and my regular account. I had also tried to create a new PayPal Developer account with a different email address, and still it would only let me create a normal PayPal account. In addition to this I found out that I had to create a PayPal Developer account to then immediately create a PayPal Sandbox account with my Developer account. I quickly looked for another API I could use, and settled on Strip. With the very brief research I did on Stripe I missed the fact they had a sandbox mode for students. Not seeing this I was under the impression that if I actually implemented Stripe further than I did there would be a real risk for someone being charged. I wish I had taken more time to find this sandbox mode as it would give a better representation of a check out system.

What I Would Do Differently

To put it bluntly I completely forgot about my design principles when starting this project. Nearly every class in my ‘core’ folder should have had some sort of interface inheritance. In this project it works but if I were to add new features down the road I would have to refactor different classes in different ways. I had also created a few classes to solve one time issues, such as ‘purchase result’, ‘listing receipt’, and ‘list view item’. If I were to re-work these classes I feel as if I could find a way to combine some of their functionality. Purchase result just says what form the cart has been purchased, listing receipt is used to show a singular transaction for an item and ultimately is only there to be displayed in the ‘order history’ section. Lastly, the listview item was for the admin app to show both listings and user accounts into one list view. There was likely a way I could have combined at least ‘listing receipt’ and ‘list view item’ into one class. The only thing else I would reconsider doing is making my project an application rather than a web based application. If I had made it a web based application I would have been able to incorporate multi-user much easier.

Upon writing this I came across a bug that I completely overlooked. If a user is to remove their listing it removes all instances of it. Even if a user has previously purchased this listing they will lose access to this information. It actually wouldn’t be very hard to fix this, all I would have to do is change the ‘deleteListing’ function in the listingDAO to make the listing unavailable rather than completely delete it. Again this is just something I realized upon reflecting on my overall project.

Future Work

I am leaving my project in a state that allows many changes, additions and reworks to be done for in the future. If I were to come back to this in the future the first thing I would attempt to do is turn it into a multi-user application. This would allow for multiple people to actively be using the app synchronously rather than just one person running it locally. For this I would require a cloud server that the application connects to constantly. For this I would need to re-work my front end into React or similar web based frameworks. As for additions I actually would like to add reviews into my application. I didn't have enough time to add this feature but I think this would have made for such a perfect application. To add reviews I would need to rework the database structure slightly and see where reviews fitted in. I would likely have a review be its own table connecting to users for the seller, and buyer and lastly a connection to the listing table. Another quick addition that I would like to add is a bidding feature. For this I wouldn't have to add a whole lot. I would just need to create an interface for general 'listings' then create a 'buy-now' listing and 'bidding-listing' each of these would essentially have the same functionality with the bidding version showing a live countdown until the item is sold, and a live price. The bidding should realistically be added after multi-user has been added though. There are a lot more quality of life changes I can add, in addition I could always improve the UI as I am not very skilled at UI/UX. Something else I would like to add is a 'cash out' option for those who have sold items. This would mean I would need to keep track of how much money a user has made from their sales. In the Sales history tab is likely where I would add this option. Once clicking this the user would likely connect to stripe or other some sort of payment service. While doing research for this project I actually found out in nearly every case of an ecommerce

platform the monetary exchange always has a middle man. For example I go to buy a listing, rather than the seller instantly getting that money I will be paying the company instead. Upon confirmation of me receiving my item will the seller be able to collect their money. This is to help stop scams from occurring as if the seller instantly had access to the money scams would be much more frequent.

Conclusion

Overall I am happy with the work that I have put into this project. While visually it's not the most appealing I think I've done a good job with the backend and what cannot be seen. Of course there are things I can improve on, but for how much time and effort I put into this project I am very satisfied with the outcome. One big self criticism I have is I definitely didn't take this project seriously enough in the beginning. Being transparent I don't think I actually started this project until a week before the first deadline (roughly 1.5 months into the project). After this deadline though I think I had a very good work effort with the project. If I had not procrastinated the beginning so much, I think my project could have been even more appealing adding in some smaller features. Through this project I was able to learn a lot about API's, CSV and .json files, common industry frameworks and how to effectively research a project model from existing sources.